Análise Sintática

Considerações sobre o modo Pânico e o conjunto de sincronização

Procedimento do Modo pânico:

```
Para cada linha do código
   Pedir o próximo token ao analisador léxico
       Se o token for o esperado
            Faz o MATCH
       Senão
            Se for o primeiro token após o último MATCH
               ou for o primeiro token da linha
                 Emite msg de erro
            Varre a frente até token de sincronização
```

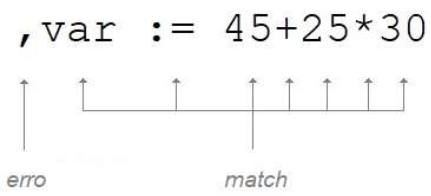
Modo Pânico

As decisões importantes a tomar nesse método de recuperação de erros são quais marcas acrescentar ao conjunto sincronizador a cada ponto da análise. Geralmente, os conjuntos de Seqüência são candidatos importantes para serem essas marcas sincronizadoras. Os conjuntos Primeiros podem também ser utilizados para evitar que o manipulador de erros descarte marcas importantes que indicam novas construções (como declarações ou expressões).

Conjunto de sincronização

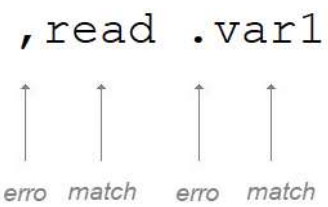
Discussão

Cenário 1:



- Caso no qual a linha é iniciada por uma sequência de tokens que não correspondem ao primeiro token de uma instrução da linguagem.
- Neste tipo de situação é necessário que o conjunto **primeiros** de símbolos **variáveis que especificam instruções** da linguagem pertençam ao conjunto de sincronização.
- Matches e mensagens de erro emitidas são adequadas.

Cenário 2:



- Caso no qual é necessário ter o conjunto primeiros como de sincronização em decorrência do erro da “,”.
- Mas após o primeiro match, considerar apenas o conjunto sequência de “read” como sendo de sincronização.
- Matchs e mensagens de erro emitidas são adequadas.

Cenário 3:

```
,var1 :=. var2 * var4
```

erro match match erro match

- Caso no qual é necessário ter o conjunto primeiros como de sincronização em decorrência do erro da “,”.
- Mas, na ocorrência do segundo erro, considerou-se apenas o conjunto sequência de “:=” como sendo de sincronização e não mais qualquer conjunto primeiros.
- Matchs e mensagens de erro emitidas são adequadas.

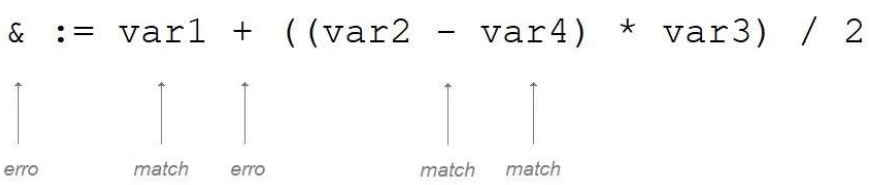
Cenário 4:

```
& := var1 + ((var2 - var4) * var3) / 2
```

erro match erro

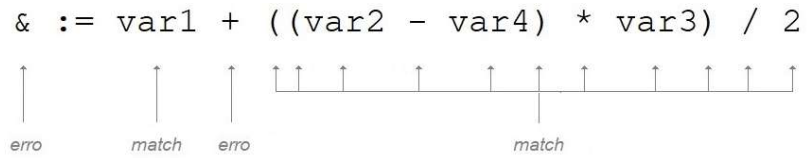
- Se após a segunda mensagem de erro, considerarmos sincronização={“:=“}, então as mensagens de erro e matchs ficam como na figura.
  - A justificativa para se utilizar apenas “:=“ e não todo o conjunto sequencia(var1) como sendo de sincronização está no fato de que o match com “var1” identifica uma instrução da linguagem (uma atribuição). Neste tipo de instrução, o único token que sucede o primeiro ID é o “:=“.
- Mensagens de erro são adequadas. Mas existe uma expressão aritmética que foi desconsiderada, mesmo estando sintaticamente correta.
- É possível uma solução alternativa que continue emitindo mensagens de erro adequadas, mas que não deixe de reconhecer (match) com toda a expressão aritmética?

Cenário 5:



- Neste caso, considerando sincronização = sequência(“var1”) = {:=, +, -,)}.
- Questão: Que ação tomar ao ler “)” após “var4”? Dois senários:
  - Não pode haver um match, pois não houve match anterior com parênteses abrindo.
  - Não deveria gerar um erro, pois “)” é um token que segundo as regras da GLC pode suceder um ID. Neste caso estaríamos indo contra o que está especificado na regra da GLC que define parênteses balanceados.

Cenário 6:



- Neste caso, (“(“) que não pertence ao conjunto sequência(“var1”), é incluído no conjunto de sincronização (“(“ ∈ Sincronização).

1. Ao ler “var1” o analisador identifica que se trata de uma instrução de atribuição.
2. Ao ler “+”, emite msg de erro, pois esperava “:=“.
3. Varre a frente em busca de um token de sincronização. Duas possibilidades:
   - 1 - ”:=“. Outros tokens do sequencia de ID não são desconsiderados pois este ID é iniciador de instrução. É plausível considerar que as demais estruturas de uma atribuição estarão presentes (“:=“ e expressão aritmética).
   - 2 - Expressão aritmética. Neste caso, os tokens de sincronização são definidos como aqueles que iniciam uma expressão (NUM, “(“, ID).

- Matches e mensagens de erro emitidas são adequadas.

Cenário 7:

- Caso tenha havido match do primeiro token da linha com o primeiro token de uma instrução da linguagem:

```
read .var1
```

match erro match

- Após emitir a msg de erro a busca será pelo token do conjunto sequência de “read” (“ID”). Esta é uma solução plausível pois a instrução já foi identificada e é pouco provável que se tenha uma expressão aritmética, o que justificaria o acréscimo de tokens ao conjunto de sincronização.

Cenário 8:

- Mais um exemplo da situação descrita o slide anterior.
- Matches e msgs de erro ocorrem de forma adequada.

```
var1 := 45+.25*30
```

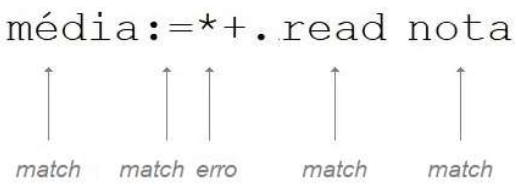
match erro match

Utilizando o conjunto primeiros para identificar outras instruções na mesma linha de código

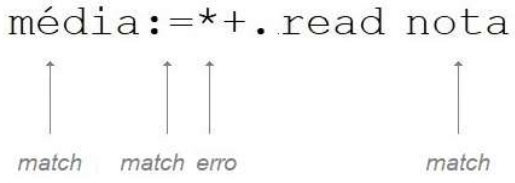
- Uma possibilidade de implementação é incluir o conjunto primeiros de todos os símbolos variáveis que especificam instruções da linguagem no conjunto de sincronização mesmo após a identificação de uma instrução, ou seja, já houve match como um token que seja o primeiro de uma instrução.
- A justificativa para isso é permitir a identificação de instruções iniciadas do decorrer de uma linha na qual já tenha havido match com uma das instruções da linguagem.

Cenários 9 e 10:

- Usando o primeiros no conjunto de sincronização:



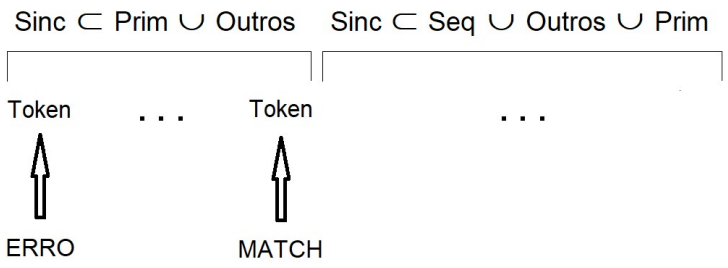
- Não usando primeiros no conjunto de sincronização:



Resumo

Definição do conjunto de sincronização em termos dos conjuntos primeiros, sequência e outros.

Caso não haja match com o primeiro token da linha:



Caso haja match com o primeiro token da linha:

